

MecâniQA – OAT 2

Ordenando a Fila

Autor: João Vitor Santos Carneiro
Arthur Lopes
Gustavo Campos
Cleyviton Ferreira
Rafael Aranha

Agenda

Objetivo da apresentação Explicar como a OAT 2 evoluiu da modelagem à persistência em CSV.

1. Modelagem:

Entidades, enums e listas dinâmicas

2. Regras:

Bloqueios de Pedido e Ordem de Serviço

3. Fila FIFO:

Ordem de chegada e execução

4. Ordenação:

Bubble Sort por nome ou código

5. Persistência:

Exportação dos dados em CSV

Contexto e objetivo da OAT 2



Problema identificado

Os arrays fixos limitavam a quantidade de peças e serviços cadastrados.

Objetivo da OAT 2

Representar o domínio da oficina com coleções dinâmicas, regras de status, fila FIFO, ordenação e arquivos CSV.

Imagem sugerida para complementar

Diagrama de fluxo de atendimento em oficina mecânica.

Modelagem dinâmica

- Entidades criadas e relacionamentos
- Cliente mantém List<Carro>
- Pedido mantém List<ItemPedido>
- OrdemServico mantém List<Servico>

```
1 package br.com.mecanica;  
2  
3 public class ItemPedido {  
4     public Peca peca;  
5     public int quantidade;  
6  
7     public ItemPedido(Peca peca, int quantidade) {  
8         this.pecas = peca;  
9         this.quantidade = quantidade;  
10    }  
11  
12    public double subtotal() {  
13        return peca.precoVenda * quantidade;  
14    }  
15  
16    public String toCsv(int codigoPedido) {  
17        return codigoPedido + ";" + peca.codigo + ";" + quantidade + ";" + subtotal();  
18    }  
19 }
```

Regras de bloqueio

- Regra principal de integridade
- Pedido FINALIZADO bloqueia novas peças
- OS fora de EM_ABERTO bloqueia alterações
- Os métodos retornam false e preservam os dados

```
public boolean adicionarServico(Servico servico) {  
    if (status != StatusOrdemServico.EM_ABERTO) {  
        System.out.println("OS fechada. Nao e possivel adicionar novos servicos.");  
        return false;  
    }  
  
    servicos.add(servico);  
    return true;  
}
```

Ciclo da Ordem de Serviço



EM_ABERTO permite alterar serviços

AGUARDANDO_EXECUCAO envia para a fila

EM_EXECUCAO indica atendimento iniciado

FINALIZADA bloqueia novas alterações

```
1 package br.com.mecanica;  
2  
3 public enum StatusOrdemServico {  
4     EM_ABERTO,  
5     AGUARDANDO_EXECUCAO,  
6     EM_EXECUCAO,  
7     FINALIZADA  
8 }
```

Fila de atendimento FIFO

```
public Queue<Servico> filaAtendimento = new LinkedList<>();
```

Queue<Servico> com LinkedList

Entrada em AGUARDANDO_EXECUCAO

Serviços entram na ordem da OS

poll() sempre remove o primeiro

```
status = StatusOrdemServico.AGUARDANDO_EXECUCAO;  
for (Servico servico : servicos) {  
    filaAtendimento.add(servico);  
}  
return true;  
}
```

Ordenação com Bubble Sort

Algoritmo implementado do zero

Enum: NOME ou CODIGO_IDENTIFI

Comparação por nome ignora
maiúsculas

Swap usa uma variável temporária

```
public void ordenarPecas(ChaveOrdenacao chave) {  
    for (int i = 0; i < pecas.size() - 1; i++) {  
        for (int j = 0; j < pecas.size() - i - 1; j++) {  
            if (compararPecas(pecas.get(j), pecas.get(j + 1), chave) > 0) {  
                Peca temporaria = pecas.get(j);  
                pecas.set(j, pecas.get(j + 1));  
                pecas.set(j + 1, temporaria);  
            }  
        }  
    }  
}
```

Persistência em CSV



Rotina estática isolada em ExportadorCsv

Arquivos usam extensão .csv

Colunas separadas por ponto e vírgula

try-with-resources trata
IOException

```
private static void escrever(Path arquivo, List<String> linhas) throws IOException {  
    try (BufferedWriter writer = Files.newBufferedWriter(arquivo, StandardCharsets.UTF_8)) {  
        for (String linha : linhas) {  
            writer.write(linha);  
            writer.newLine();  
        }  
    }  
}
```

Testes e validação



Cenário 1 - Bloqueio do Pedido

Dado um Pedido com status FINALIZADO

Quando adicionarItem() for chamado

Então o método retorna false e a lista permanece igual

Cenário 2 - Ordem FIFO

Dado dois serviços inseridos na fila

Quando executarProximoServico() for chamado

Então o primeiro serviço inserido é removido

Resultados finais



- O domínio agora usa entidades e coleções dinâmicas.
- As regras de status protegem Pedidos e Ordens de Serviço.
- A fila FIFO garante a ordem correta de atendimento.
- Bubble Sort organiza peças e serviços por nome ou código.
- A exportação CSV preserva os dados fora da memória.

Referências



- Documento OAT 2 – Ordenando a Fila
- Caderno de Guias – OAT 2 – Tutoriais
- GitHub: [Unex-Estudos/mecaniQA-dados-rio-de-janeiro](#)
- Java SE: List, ArrayList, Queue, LinkedList e java.nio.file